

# RAPPI

## Caso Técnico: Sistema de Alertas Operacionales con AI

Rol: AI Engineer

Tiempo de entrega: 5 días calendario • Presentación: 30 minutos (20 demo + 10 Q&A)

### 1. Contexto del Problema

Rappi opera flotas de repartidores en tiempo real en cientos de zonas a través de 9 países. El equipo de Operations depende de la disponibilidad de repartidores conectados para atender la demanda de órdenes de forma sostenible. Cuando la oferta de repartidores y la demanda de órdenes no están balanceadas, se generan dos tipos de problemas:

- Saturación: demasiadas órdenes relativas a repartidores disponibles — clientes esperan más tiempo, órdenes se pierden, NPS cae.
- Sobre-oferta: demasiados repartidores activos con pocas órdenes — ineficiencia operacional, costo sin retorno.

La métrica central de balance operacional es:

$Ratio = \text{Órdenes} / \text{Repartidores Conectados}$  < 0.5: Sobre-oferta (ineficiencia de costo) • 0.9 – 1.2: Rango saludable • > 1.8: SATURACIÓN (pérdida de órdenes)

El equipo de Operations actualmente gestiona el balance de forma reactiva. Tu misión es construir un sistema que lo haga de forma proactiva: detectar condiciones que degradarán la operación antes de que ocurran, y enviar alertas accionables con recomendaciones concretas al equipo a través de Telegram.

El caso está dividido en tres módulos progresivos. El Módulo 1 es el fundamento analítico: sin él, los Módulos 2 y 3 no tienen base. Prioriza en ese orden.

## 2. Entregables Requeridos

### Módulo 1 — Diagnóstico Operacional (25%)

Analiza el dataset histórico e identifica los factores que afectan la operación. Tu análisis debe responder a estas cinco preguntas de negocio, presentadas de menor a mayor sofisticación:

1. **P1.** ¿En qué horas del día y en qué zonas la operación alcanza niveles críticos de saturación? Cuantifica.
2. **P2.** ¿Qué variable externa del dataset se correlaciona con el deterioro del ratio operacional? Describe el mecanismo.
3. **P3.** ¿Todas las zonas responden igual a esa variable? Identifica las más vulnerables y explica por qué tienen mayor sensibilidad.
4. **P4.** ¿El nivel de earnings (incentivos) está bien calibrado a lo largo del mes? ¿Detectas periodos con gasto ineficiente? Muestra los días exactos.
5. **P5.** ¿Qué relación tiene el nivel de earnings con la saturación operacional? ¿Es una relación simple o depende de otras condiciones?

*Nota metodológica: se evaluará no solo qué encuentras, sino cómo llegas a las conclusiones. Un número sin contexto no es un hallazgo. Una correlación sin validación de causalidad o sin control de variables confusas es una conclusión incompleta.*

### Entregables Módulo 1

- Notebook o script reproducible con el análisis completo (Python recomendado).
- Visualizaciones para cada hallazgo (el gráfico correcto para cada tipo de pregunta).
- Resumen de hallazgos en máximo 1 página: patrones encontrados + impacto cuantificado en la operación.

## Módulo 2 — Motor de Alertas Tempranas con API de Clima (35%)

Basado en los patrones que identificaste en el Módulo 1, construye un sistema que monitoree las condiciones climáticas en tiempo real y genere alertas cuando se detecte riesgo operacional inminente.

### 2a. Integración climática y mapeo de zonas

Conecta una weather API pública para obtener forecast horario de precipitación para las 14 zonas operacionales de Monterrey. El dataset incluye en la hoja ZONE\_POLYGONS los polígonos geográficos de cada zona en formato WKT. Debes usar estos polígonos para mapear coordenadas del API a la zona operacional correcta.

APIs recomendadas (todas gratuitas para el volumen de este caso):

- Open-Meteo (open-meteo.com) — sin API key, cubre Monterrey, devuelve precipitation en mm/hr por coordenada. Formato de respuesta compatible con el dataset.
- WeatherAPI.com — requiere registro gratuito, forecast horario hasta 3 días.
- Open Weather Map — requiere registro gratuito, ofrece current + forecast horario.

Documenta por qué elegiste el API que elegiste.

### 2b. Motor de decisión

El corazón del sistema. Debes diseñar las reglas que determinan cuándo y cómo alertar. Cada decisión de diseño debe estar justificada con los datos históricos del Módulo 1. Estas son las preguntas que tu motor debe resolver:

- ¿Qué umbral de precipitación (mm/hr) dispara una alerta? ¿Es el mismo umbral para todas las zonas?
- ¿Con cuánta anticipación actuar? (forecast 1h vs 3h: trade-off entre precisión y ventana para reaccionar)
- ¿Cuánto debe subir el earnings recomendado? Debe ser un número específico estimado desde el histórico, no un rango genérico.
- ¿Cómo evitas mandar la misma alerta dos veces por el mismo evento?

### 2c. Script funcional

Un script ejecutable que, al correrlo en este momento, produzca output del tipo:

```
Zona: SantiagoPrecipitación esperada: 7.2 mm/hr en las próximas 2 horasRiesgo: ALTO (ratio proyectado ~1.9 basado en histórico)Acción recomendada: subir earnings de 55 a 78 MXN en los próximos 30 minZonas secundarias a monitorear: Carretera Nacional, Santa Catarina
```

### Entregables Módulo 2

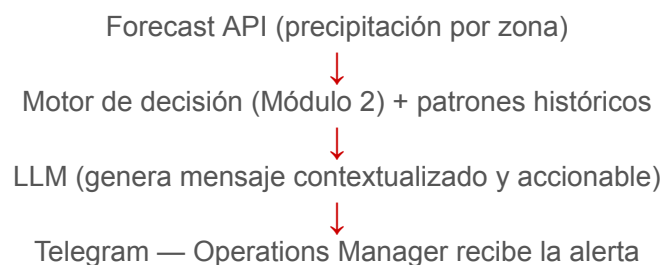
- Código funcional del motor de alertas.

- Documento de 1 página justificando los umbrales y reglas del motor con evidencia de los datos históricos.

## Módulo 3 — Agente AI con Notificaciones Telegram (40%)

Integra un LLM al pipeline del Módulo 2 para que las alertas se generen en lenguaje natural y se envíen automáticamente a un canal de Telegram. El objetivo es que un Operations Manager reciba un mensaje que pueda leer en 10 segundos y saber exactamente qué hacer.

### Arquitectura del flujo



### El mensaje que el agente debe generar

El candidato diseña el prompt. El mensaje de Telegram resultante debe contener:

- Zona afectada y nivel de riesgo (bajo / medio / alto / crítico).
- Qué se espera que pase, basado en eventos históricos similares del dataset.
- Acción concreta con número específico — no “considerar subir incentivos” sino “subir earnings de X a Y MXN”.
- Ventana de tiempo para actuar antes de que la saturación ocurra.
- Zonas secundarias que podrían verse afectadas.

*Criterio de evaluación del mensaje: ¿un Operations Manager en campo podría leerlo en 10 segundos y saber exactamente qué hacer? Si la respuesta es sí, el mensaje está bien diseñado.*

### Preguntas de arquitectura para la presentación

Prepara respuestas para estas preguntas. Las responderás durante el Q&A:

- ¿Cómo maneja tu sistema los falsos positivos? (lluvia pronosticada que no llega)
- ¿Cómo evitas alert fatigue? (demasiadas notificaciones = se ignoran)
- ¿Cómo escalarías este sistema a otras ciudades de Rappi? El dataset tiene solo Monterrey, pero Rappi opera en cientos de zonas.

## Bonus (no obligatorio)

- Memoria del agente: que no reenvíe una alerta por la misma zona dentro de una ventana de tiempo configurable.
- Nivel de alerta diferenciado por zona según la sensibilidad detectada en el Módulo 1.
- Resumen diario automático: un mensaje consolidado al final del día con los eventos ocurridos y su impacto real vs proyectado.

## Entregables Módulo 3

- Agente AI funcional que envía mensajes reales a un canal de Telegram.
- Demo en vivo durante la presentación (no video grabado): el sistema debe enviar al menos un mensaje a Telegram en tiempo real.
- README con instrucciones de reproducción y costo estimado de API por uso.

## 3. Dataset Proporcionado

Recibirás un archivo Excel (rappi\_delivery\_case\_data.xlsx) con tres hojas:

### Hoja RAW\_DATA — Dataset operacional principal

30 días de datos históricos por hora, para 14 zonas operacionales de Monterrey. El dataset contiene 10,080 filas.

| Columna          | Tipo    | Descripción                                                        |
|------------------|---------|--------------------------------------------------------------------|
| COUNTRY          | string  | País de operación                                                  |
| DATE             | string  | Fecha en formato YYYY-MM-DD                                        |
| HOURL            | integer | Hora del día (0–23, hora local Monterrey / UTC-6)                  |
| CITY             | string  | Ciudad                                                             |
| ZONE             | string  | Zona operacional (14 zonas, ver ZONE_INFO)                         |
| CONNECTED_RT     | integer | Repartidores conectados y disponibles en esa hora                  |
| ORDERS           | integer | Pedidos recibidos en esa zona-hora                                 |
| EARNINGS         | float   | Pago por pedido en MXN — refleja el nivel de incentivo activo      |
| PRECIPITATION_MM | float   | Precipitación en mm/hr — mismo formato que Open-Meteo y WeatherAPI |

### Hoja ZONE\_INFO — Centroides de zonas

Coordenadas del centroide de cada zona (calculadas automáticamente desde los polígonos reales). Útiles para llamadas al weather API.

| Columna          | Descripción                                 |
|------------------|---------------------------------------------|
| ZONE             | Nombre de la zona operacional               |
| LATITUDE_CENTER  | Latitud del centroide del polígono (WGS84)  |
| LONGITUDE_CENTER | Longitud del centroide del polígono (WGS84) |
| DESCRIPTION      | Descripción geográfica de la zona           |

## Hoja ZONE\_POLYGONS — Límites geográficos

Polígonos reales de cada zona en formato WKT (Well Known Text), coordenadas en EPSG:4326 (longitud primero, luego latitud). Necesarios para el Módulo 2: dado un par lat/lon del weather API, usa un point-in-polygon check para identificar a qué zona operacional corresponde.

```
Ejemplo Python — asignar coordenada a zona:
from shapely.geometry import Point
from shapely import wkt
point = Point(-100.31, 25.67) # Point(longitud, latitud)
for zone_name, geom_wkt in zone_geometries.items():
    if wkt.loads(geom_wkt).contains(point):
        print(zone_name)
```

## 4. Requisitos Técnicos

### Stack tecnológico

Libertad total de herramientas. Usa lo que mejor conozcas y puedas defender. Algunos ejemplos:

- LLMs: OpenAI (GPT-4o, GPT-4), Anthropic (Claude), Google (Gemini), modelos open-source (Llama, Mistral).
- Lenguajes: Python (recomendado para el análisis), JavaScript/TypeScript, o cualquier otro.
- Orquestación: LangChain, LlamaIndex, n8n, Make, o implementación propia.
- Notificaciones: Telegram Bot API (obligatorio para Módulo 3).

Si usas APIs pagas, documenta el costo estimado por uso (ej: “~\$0.03 por alerta generada con GPT-4o”).

### Código y reproducibilidad

Incluye un README con:

- Instrucciones para reproducir los tres módulos paso a paso.
- Variables de entorno o API keys necesarias (con placeholder, nunca la key real).
- Cómo configurar el bot de Telegram para la demo.
- Cualquier dependencia de setup (conda env, Docker, etc.).

### Buenas prácticas (valoradas pero no obligatorias)

- Manejo de errores robusto (qué pasa si el weather API no responde).
- Logging que permita auditar qué alertó el sistema, cuándo y por qué.

- Código limpio y organizado — el candidato debe poder explicar cualquier bloque en la demo.

## 5. Criterios de Evaluación

Tu solución será evaluada con la siguiente rúbrica (100 puntos):

| Criterio                     | Peso | Qué evaluamos                                                                                                           |
|------------------------------|------|-------------------------------------------------------------------------------------------------------------------------|
| Patrones encontrados         | 15%  | Checklist de 5 hallazgos: cuántos identificó, con qué precisión y con qué cuantificación.                               |
| Rigor metodológico           | 10%  | ¿Segmentó correctamente? ¿Controló variables confusas? ¿Evitó conclusiones incorrectas de correlaciones simples?        |
| Motor de decisión            | 15%  | Los umbrales y reglas están justificados con los datos históricos. Las recomendaciones son específicas y cuantificadas. |
| Sistema funcional            | 20%  | Demo en vivo sin errores. El código corre en un entorno limpio siguiendo el README.                                     |
| Calidad del agente AI        | 15%  | El mensaje de Telegram es accionable en 10 segundos. El prompt está bien diseñado para el contexto operacional.         |
| Arquitectura y escalabilidad | 15%  | Justificación de decisiones técnicas. Respuestas claras a las preguntas de escalabilidad del Q&A.                       |
| Documentación y presentación | 10%  | README claro y reproducible. Capacidad de explicar cada decisión durante la demo.                                       |

### Lo que buscamos en un candidato excepcional

- **Coherencia end-to-end:** Conexión analítica-operacional: los hallazgos del Módulo 1 se ven reflejados directamente en las reglas del Módulo 2. El sistema no es genérico — está calibrado para esta operación.
- **Criterio de negocio:** Reglas específicas justificadas con números del dataset, no intuición o valores arbitrarios.
- **Pensamiento crítico:** No solo resolver el problema, sino identificar limitaciones y proponer qué harían con más tiempo o datos.
- **Orientación al usuario:** Un ops manager de campo usaría este sistema. La interfaz y el mensaje importan tanto como el modelo.

## 6. Formato de Entrega y Presentación

### Repositorio

Sube tu solución a un repositorio Git (GitHub o GitLab) público o con acceso compartido. Estructura sugerida:

```
rappi-case/├── modulo1_diagnostico/  # Notebook + visualizaciones├── modulo2_motor_alertas/  # Script del motor de decisión└── modulo3_agente_telegram/  # Agente AI + integración Telegram└── README.md  # Instrucciones de reproducción
```

### Presentación (30 minutos)

Estructura sugerida:

| Sección             | Tiempo | Contenido                                                         |
|---------------------|--------|-------------------------------------------------------------------|
| Contexto y approach | 3 min  | ¿Cómo interpretaste el problema? ¿Qué priorizaste y por qué?      |
| Demo Módulo 1       | 8 min  | Walk-through del análisis: muestra cómo llegaste a cada hallazgo. |
| Demo Módulo 2       | 5 min  | Muestra el motor corriendo con datos reales del weather API.      |
| Demo Módulo 3       | 5 min  | Demo en vivo: el agente envía un mensaje real a Telegram.         |
| Decisiones técnicas | 4 min  | Arquitectura, elección de LLM, trade-offs principales.            |
| Limitaciones        | 2 min  | ¿Qué mejorarías con más tiempo o datos?                           |
| Q&A                 | 10 min | Preguntas del panel, incluidas las de arquitectura del Módulo 3.  |

*Importante: la demo del Módulo 3 debe ser en vivo, no un video grabado. Queremos ver el sistema enviando un mensaje a Telegram en tiempo real durante la presentación.*

## 7. Preguntas Frecuentes

### ¿Puedo hacer preguntas sobre el dataset o las métricas?

El dataset está diseñado para ser autoexplicativo con el diccionario de columnas. Si tienes dudas conceptuales sobre la operación de delivery, puedes escribir al contacto indicado. No respondemos preguntas técnicas de implementación.

### ¿Necesito completar los tres módulos?



El Módulo 1 es obligatorio como fundamento. Los Módulos 2 y 3 son el diferenciador. Un Módulo 1 sólido con un Módulo 2 incompleto pero bien fundamentado vale más que tres módulos superficiales. Prioriza calidad sobre completitud.

¿Qué LLM debo usar?

El que mejor conozcas y puedas defender. Si usas modelos de pago, documenta el costo estimado. Si prefieres modelos gratuitos u open-source, es completamente válido.

¿Necesito deployment en la nube?

No. Con que funcione localmente es suficiente. Si haces deployment (Railway, Render, Fly.io, etc.) es un plus que se mencionará en la evaluación, pero no cambia el peso de los criterios.

¿Puedo usar herramientas no-code como n8n o Make?

Sí para la orquestación del Módulo 3. Para el Módulo 1 se espera código (notebook o script) para poder evaluar el razonamiento analítico. Si usas no-code, exporta el workflow e incluye instrucciones de importación.

8. Timeline y Contacto

| Hito                                  | Fecha                                |
|---------------------------------------|--------------------------------------|
| Envío del caso y dataset              | [Día 0 — fecha de envío]             |
| Fecha límite de entrega (repositorio) | [Día 5 — 5 días desde envío]         |
| Presentación                          | [A coordinar tras la entrega]        |
| Contacto para dudas conceptuales      | Eduardo Daniel Chain Francisco Gomez |

9. Consejos Finales

- **Empieza por el diagnóstico:** El Módulo 1 es el fundamento. Si no encuentras los patrones correctos, los módulos siguientes no tendrán base sólida. Invértelo el tiempo necesario.
- **Los números importan:** Un motor con reglas simples y bien fundamentadas supera a un modelo complejo con umbrales arbitrarios. Los números deben poder justificarse con el dataset.
- **Piensa en el usuario:** Diseña el mensaje de Telegram pensando en un Operations Manager que lo recibe a las 7pm en medio de una operación activa. ¿Entendería qué hacer?
- **Gestiona tu tiempo:** 5 días pasan rápido. Módulo 1 (día 1-2) → Módulo 2 (día 3) → Módulo 3 (día 4) → README + presentación (día 5).
- **Documenta tus decisiones:** Cualquier decisión técnica puede ser correcta si la puedes defender con datos y razonamiento. Lo que evaluamos no es qué elegiste, sino por qué.

¡Mucha suerte! Estamos emocionados de ver tu solución.

*Los datos han sido generados sintéticamente y no representan operaciones reales de Rappi.*